

Experiment No. 10

Aim: Evaluation of Classification Algorithms.

Theory:

The Supervised Machine Learning algorithm can be broadly classified into Regression and Classification Algorithms. In Regression algorithms, we have predicted the output for continuous values, but to predict the categorical values, we need Classification algorithms.

What is the Classification Algorithm?

The Classification algorithm is a Supervised Learning technique that is used to identify the category of new observations on the basis of training data. In Classification, a program learns from the given dataset or observations and then classifies new observation into a number of classes or groups. Such as, **Yes or No, 0 or 1, Spam or Not Spam, cat or dog**, etc. Classes can be called as targets/labels or categories.

In classification algorithm, a discrete output function(y) is mapped to input variable(x).

$Y = f(x)$, where Y = categorical output

The best example of an ML classification algorithm is **Email Spam Detector**.

The main goal of the Classification algorithm is to identify the category of a given dataset, and these algorithms are mainly used to predict the output for the categorical data.

Classification algorithms can be better understood using the below diagram. In the below diagram, there are two classes, class A and Class B. These classes have features that are similar to each other and dissimilar to other classes.

Types of ML Classification Algorithms:

Classification Algorithms can be further divided into the Mainly two category:

Linear Models

- o Logistic Regression
- o Support Vector Machines

Non-linear Models

- o K-Nearest Neighbours
- o Kernel SVM
- o Naïve Bayes
- o Decision Tree Classification
- o Random Forest Classification

Evaluating a Classification model:

Once our model is completed, it is necessary to evaluate its performance; either it is a Classification or Regression model. So, for evaluating a Classification model, we have the following ways:

1. Log Loss or Cross-Entropy Loss:

It is used for evaluating the performance of a classifier, whose output is a probability value between the 0 and 1. For a good binary Classification model, the value of log loss should be near to 0. The value of log loss increases if the predicted value deviates from the actual value. The lower log loss represents the higher accuracy of the model. For Binary classification, cross-entropy can be calculated as:

2. Confusion Matrix:

The confusion matrix provides us a matrix/table as output and describes the performance of the model. It is also known as the error matrix. The matrix consists of predictions result in a summarized form, which has a total number of correct predictions and incorrect predictions. The matrix looks like as below table:

	Actual Positive	Actual Negative
Predicted Positive	True Positive	False Positive
Predicted Negative	False Negative	True Negative

From the confusion matrix, we can find the following metrics

Accuracy: Accuracy is used to measure the performance of the model. It is the ratio of Total correct instances to the total instances.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{TN} + \text{FN})$$

Precision: Precision is a measure of how accurate a model's positive predictions are. It is defined as the ratio of true positive predictions to the total number of positive predictions made by the model.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall: Recall measures the effectiveness of a classification model in identifying all relevant instances from a dataset. It is the ratio of the number of true positive (TP) instances to the sum of true positive and false negative (FN) instances.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-Score: F1-score is used to evaluate the overall performance of a classification model. It is the harmonic mean of precision and recall,

$$\text{F1-Score} = (2 * \text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

3. AUC-ROC Curve: ROC curve stands for **Receiver Operating Characteristics Curve** and AUC stands for **Area Under the Curve**.

It is a graph that shows the performance of the classification model at different thresholds. To visualize the performance of the multi-class classification model, we use the AUC-ROC Curve. The ROC curve is plotted with TPR and FPR, where TPR (True Positive Rate) on Y axis and FPR (False Positive Rate) on X-axis.

Input:

Step 1: Importing Necessary Packages

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, confusion_matrix, classification_report, roc_curve, auc
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load the Iris Dataset

```
[2]: # Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Feature scaling (important for many ML algorithms)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Show the shape of the data
print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

Training data shape: (105, 4)
Testing data shape: (45, 4)
```

Step 3: Train the Classifier Model

```
[3]: # Initialize the classifier
      clf = RandomForestClassifier(random_state=42)

      # Train the model on the training data
      clf.fit(X_train, y_train)

      # Predict on the test data
      y_pred = clf.predict(X_test)
```

Step 4: Prediction To Find (Accuracy, Precision, Recall, F1 Score, Confusion Matrix, Classification Report, ROC-AUC for Binary Classification)

```
[12]: accuracy = accuracy_score(y_test, y_pred)
      print(f"Accuracy: {accuracy:.4f}")

      precision = precision_score(y_test, y_pred, average='weighted')
      print(f"Precision: {precision:.4f}")

      recall = recall_score(y_test, y_pred, average='weighted')
      print(f"Recall: {recall:.4f}")

      f1 = f1_score(y_test, y_pred, average='weighted')
      print(f"F1 Score: {f1:.4f}")

      cm = confusion_matrix(y_test, y_pred)
      print("Confusion Matrix:")
      print(cm)

      report = classification_report(y_test, y_pred, target_names=iris.target_names)
      print("Classification Report:")
      print(report)

      # ROC-AUC (for binary classification, or one-vs-rest approach for multi-class)
      y_prob = clf.predict_proba(X_test) # predict_proba is used to get the probability estimate
      roc_auc = roc_auc_score(y_test, y_prob, multi_class='ovr')
      print(f"ROC-AUC: {roc_auc:.4f}")
```

```
Accuracy: 0.8889
Precision: 0.8981
Recall: 0.8889
F1 Score: 0.8878
Confusion Matrix:
[[15  0  0]
 [ 0 14  1]
 [ 0  4 11]]
Classification Report:
              precision    recall  f1-score   support

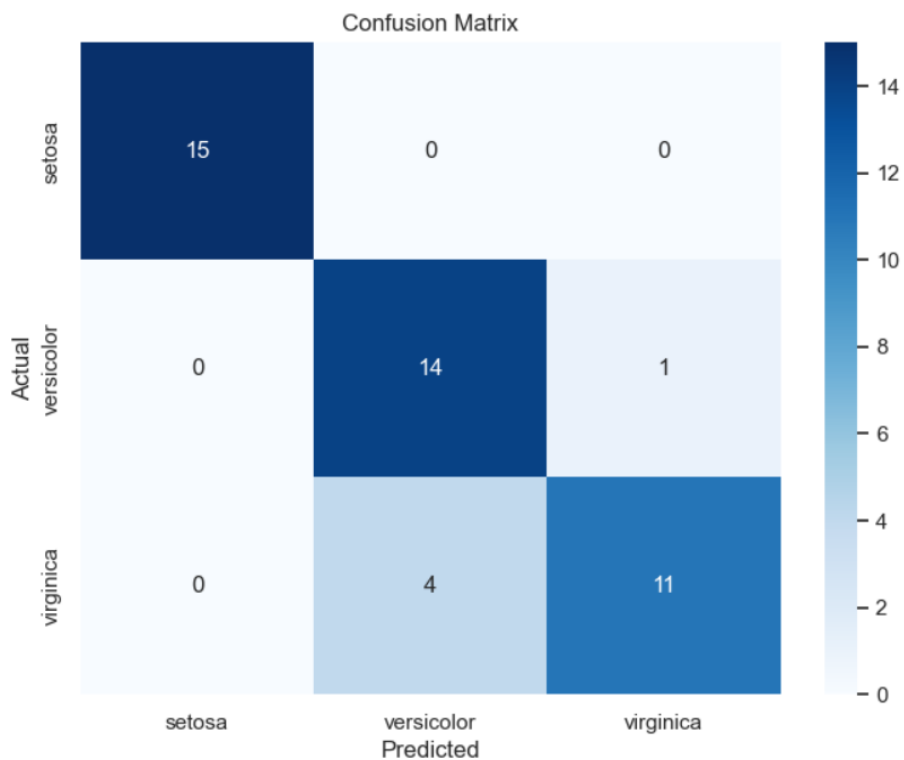
   setosa         1.00        1.00        1.00        15
  versicolor    0.78         0.93         0.85        15
   virginica    0.92         0.73         0.81        15

   accuracy                    0.89         45
  macro avg              0.90         0.89         0.89         45
 weighted avg              0.90         0.89         0.89         45

ROC-AUC: 0.9926
```

Step 5: Plotting and Visualizing the Confusion Matrix

```
[5]: # Visualizing the confusion matrix using Seaborn
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=iris.target_names, yticklabels=iris.target_names,
            plt.xlabel('Predicted')
            plt.ylabel('Actual')
            plt.title('Confusion Matrix')
            plt.show()
```



Step 6: Define the parameter grid for the best score

```
[7]: # Define the parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

# Initialize GridSearchCV
grid_search = GridSearchCV(estimator=clf, param_grid=param_grid, cv=5, n_jobs=-1, verbose=2)

# Fit GridSearchCV
grid_search.fit(X_train, y_train)

# Print the best parameters and the best score
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Best Cross-Validation Score: {grid_search.best_score_:.4f}")
```

Fitting 5 folds for each of 216 candidates, totalling 1080 fits

C:\Users\meets\AppData\Local\Programs\Python\Python312\Lib\site-packages\numpy\ma\core.py:2820: RuntimeWarning: invalid value encountered in cast
_data = np.array(data, dtype=dtype, copy=copy,

Best Parameters: {'bootstrap': True, 'max_depth': None, 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 100}
Best Cross-Validation Score: 0.9619

Step 7: Find the Best Score Between Initial and New Model

```
[11]: print("Initial Model vs Best Model Performance Comparison:")
print(f"Initial Accuracy: {accuracy:.4f} vs Best Accuracy: {best_accuracy:.4f}")
print(f"Initial ROC-AUC: {roc_auc:.4f}")
```

Initial Model vs Best Model Performance Comparison:
Initial Accuracy: 0.8889 vs Best Accuracy: 0.9111
Initial ROC-AUC: 0.9926

Conclusion: Thus, we have successfully studied and implemented the evaluation of classification Algorithm by training the model for better performance